

Chess Engine Application

HSC Software Engineering Major Project

Made by: Jeremy Nguyen
Teacher: Mr Zhukov
Tools: Python, Pygame, VS Code, GitHub



**CHECKMATE LOGIC
MEETS SOFTWARE ENGINEERING**





Why build a chess engine?

The need

A school-friendly chess engine needs to enforce official chess rules while staying simple enough for students to use and understand.

- Users: students, chess club members, teachers and tournament organisers
- Need: legal moves, fair turn control, check and checkmate detection
- Benefits: practise rules, test strategy, play against AI and review moves
- History: chess moved from traditional boards into online play and engines



**Chess is rule-heavy.
Software must be exact.**

Traditional chess



Digital engine

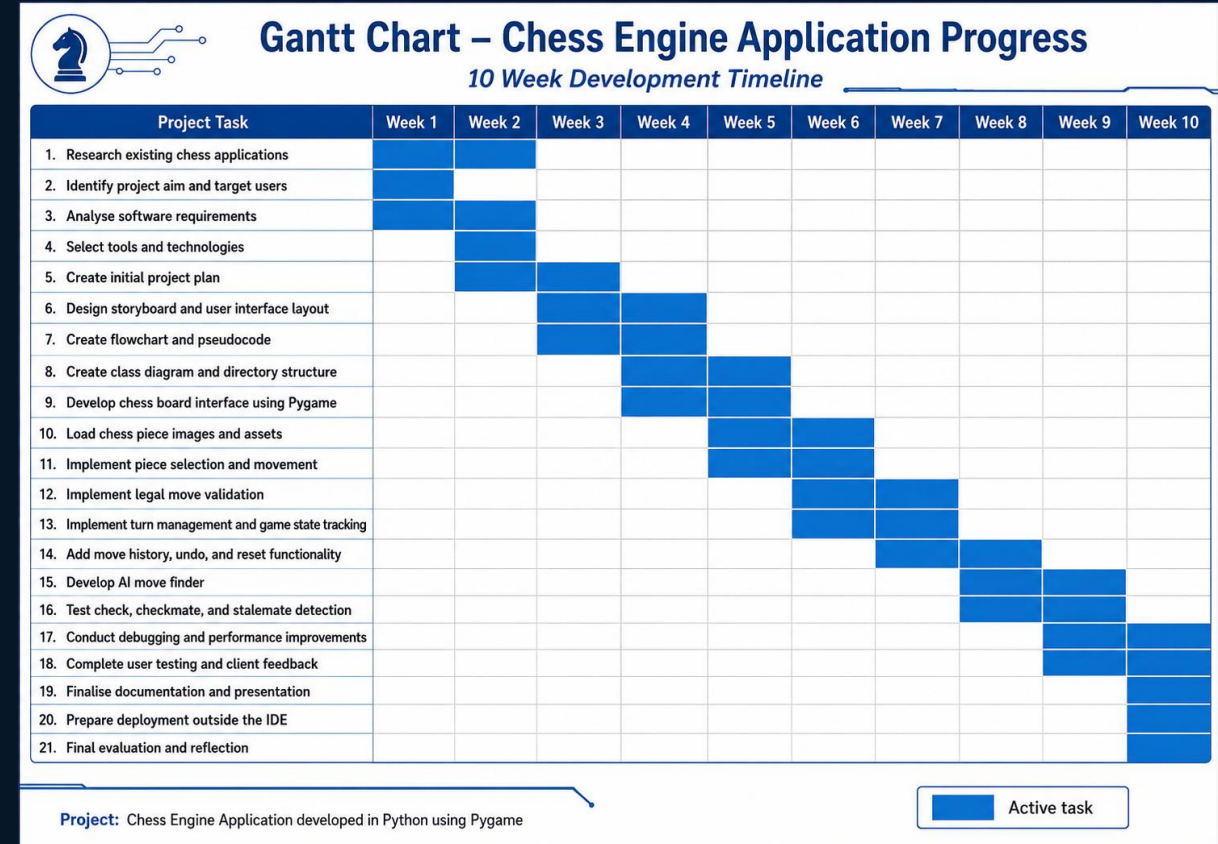
- | | |
|-----------------------|------------------------|
| ● Physical board | ● Automated validation |
| ● Human rule checking | ● AI opponent |
| ● Limited review | ● Move history |



WAgile development plan

Why WAgile?

- Waterfall supported planning, requirements, risks and diagrams before coding
- Agile supported feature-by-feature development and testing
- DevOps practices supported GitHub commits, backup and regular debugging
- This gave the project structure while still allowing improvements during development



1 Research +
requirements

2 Board + GUI

3 Legal moves

4 AI + history

5 Testing + docs



Technology choices and justification

Python

Readable language for OOP, algorithms and fast development.

Pygame

Handles board drawing, image loading, mouse input and screen updates.

VS Code

Supports Python extensions, terminal use, debugging and file organisation.

GitHub

Tracks commits, stores backups and shows development progress.

Figma

Supports early interface planning and layout decisions.

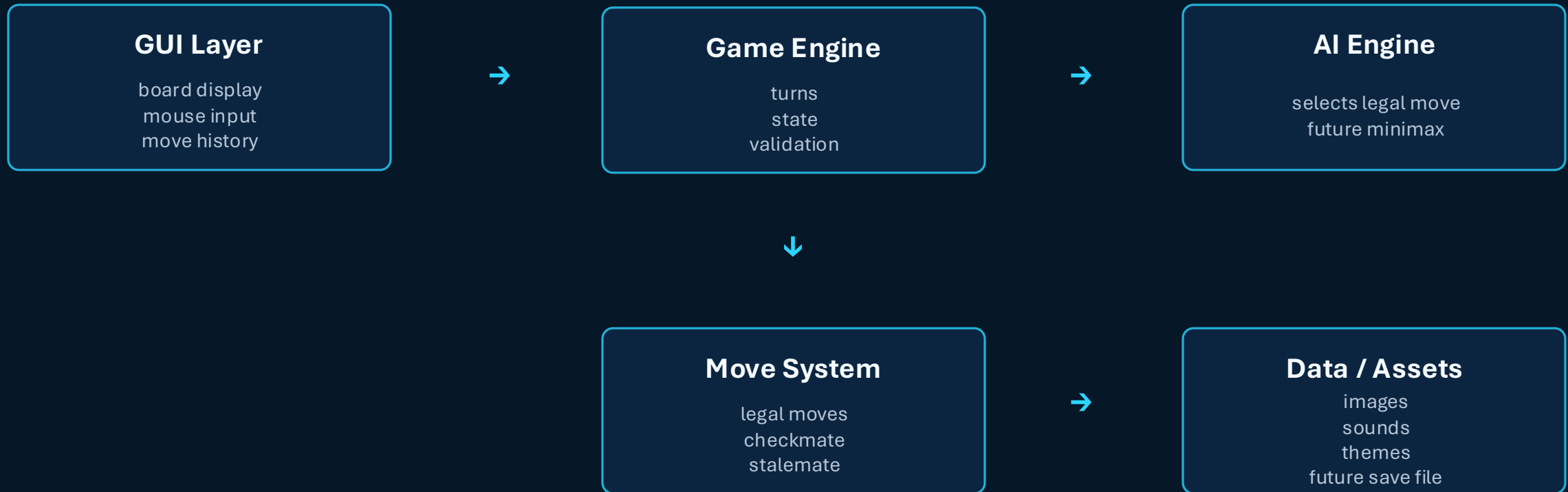
Multiprocessing

Helps AI calculations avoid freezing the main game loop.

Evaluation decision: Python + Pygame was selected over Java and NodeJS because the project needed fast desktop GUI development, clear logic and a manageable Year 12 scope.

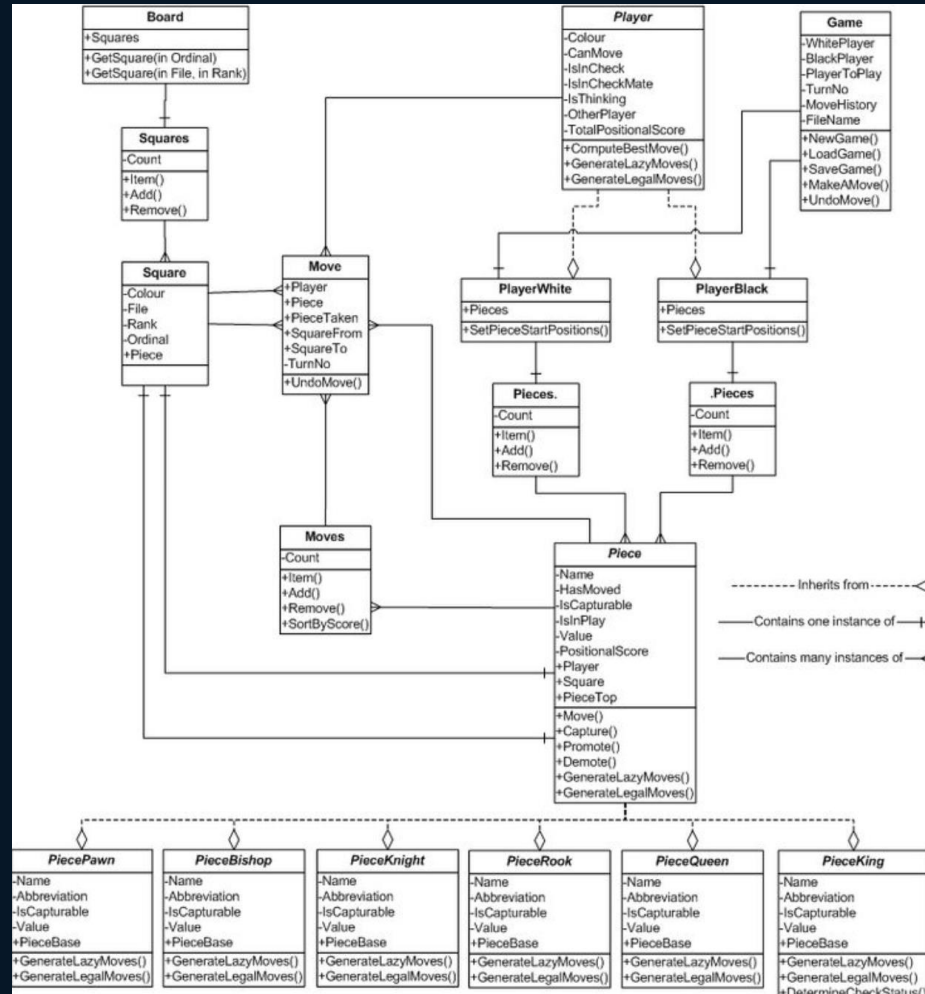


High-level system architecture



Why this matters: each module has one main job, which makes the project easier to test, debug and extend.

Class diagram and system interactions



Core classes

- Game controls the main loop, reset and undo
- Board stores squares and current layout
- Move stores start, end and captured piece data
- Piece is the parent class for all chess pieces
- Pawn, Rook, Knight, Bishop, Queen and King apply specific movement rules
- Player tracks colour, legal moves and check status

UML was used before coding to reduce confusion and plan how objects would communicate.



Security controls and limitations

Desktop current version

- Mouse input is controlled through board coordinates
- Illegal moves are rejected before the board updates
- Exception handling reduces crashes from missing files
- No user accounts or online database in current version

Needed for future online version

- Authentication for user accounts and saved games
- Password hashing instead of plain text passwords
- CSRF protection for web forms
- XSS protection for usernames, chat and inputs
- Database query protection for saved results

Security limitation: features such as authentication, hashing, CSRF and XSS controls are not required in the desktop prototype, but they become essential if the project expands into an online chess platform.



Testing strategy and edge cases

Unit testing
piece movement
legal move checks

Integration testing
GUI + engine
AI + move system

System testing
full games
reset + undo

User testing
clarity
responsiveness

Edge cases tested

- Check, checkmate and stalemate
- Illegal moves and turn order
- Undo, reset and move history accuracy
- Pawn promotion, castling and en passant
- AI must select legal moves only



Recommended evidence: pytest output, gameplay screenshots, accessibility checks, checkmate test, illegal move test.



Version control evidence

Commits on Jun 22, 2026

Update .DS_Store

 jeremycodingfr committed last week

21c9f16

Commits on Jun 21, 2026

Update ai.py

 jeremycodingfr committed last week

d8af0c8

Commits on Jun 20, 2026

Update ai.py

 jeremycodingfr committed last week

9453d01

Updated ai.py

 jeremycodingfr committed last week

f6079e4

Updated code

 jeremycodingfr committed last week

d7e92eb

Commits on Jun 19, 2026

Update move.py

 jeremycodingfr committed 2 weeks ago

474fd41

Commits on Jun 15, 2026

Changed image 

 jeremycodingfr committed 2 weeks ago

fde119d

Image

 jeremycodingfr committed 2 weeks ago

888d4b9

Commits on Jun 14, 2026

Evidence to discuss

- Commits across the development timeline
- Separate commits for board, legal moves, AI and testing
- Branches for experimental features
- Issues used to track bugs and fixes
- Pull requests useful for code review in a team project

Why it matters: GitHub proves the project was built over time and protects the code from accidental loss.



Demonstration script

Run command

```
python src/main.py
```

- 1 Open the project and launch the game window
- 2 Move a legal piece and show board update
- 3 Attempt an illegal move and show it is rejected
- 4 Show AI selecting a legal move
- 5 Use move history, undo and reset
- 6 Demonstrate check or checkmate if prepared

Demonstration goal: prove the product works as a real software solution, not only as documentation.



Development challenges

Move legality

Chess rules are strict. The program had to reject illegal moves, protect the king and keep the game state accurate.

Performance

Animations, move history and AI calculations had to run smoothly during gameplay.

AI complexity

The AI needed to choose only legal moves without making the interface slow or unreliable.

Debugging

A small board-state bug could affect captures, checks and later moves.

Main response: use modular code, staged testing, GitHub commits and targeted debugging after each feature.



Next development stage



Highest priority: improve AI strength with minimax, then alpha-beta pruning to keep calculation time manageable.



Final reflection

What I learned

- Planning gives a project direction before coding begins
- Object-oriented design makes complex systems easier to manage
- Testing must happen throughout development, not only at the end
- GitHub gives backup, history and development evidence
- Software engineering turns a game idea into a reliable product

Final outcome

The final application met the main requirements by providing a playable graphical chess game with legal move validation, move history, undo, reset and an AI opponent. The project also showed evidence of methodology, design, risk management, testing, version control and evaluation.

Overall: the project demonstrates the full software development lifecycle through a practical chess engine.